

W4111_003_2024_3 – Introduction to Databases

Homework 3A

Storage Devices

S1. What are three important criteria for choosing the type of storage to use for data?

1. **Capacity:** The amount of data that needs to be stored is a crucial factor in selecting the type of storage. Different storage options have varying capacities, and choosing the right one ensures that all data can be accommodated without running out of space.
2. **Performance:** The speed at which data can be accessed and retrieved is critical for many applications. Storage options like SSDs offer faster performance compared to traditional HDDs, making them suitable for applications that require rapid data access.
3. **Cost:** The cost of storage is a significant consideration, especially for large-scale data storage needs. Different storage options have varying price points, and choosing the most cost-effective option that meets the capacity and performance requirements is essential.

S2. Some databases use magnetic disks in a way that only sectors in outer tracks are used while sectors in inner tracks are left unused. What might be the benefits of doing so?

1. **Improved Performance:** Data stored in the outer tracks of a disk can be accessed faster than data in the inner tracks. By utilizing only the outer tracks, databases can achieve better read and write performance.
2. **Reduced Wear and Tear:** The outer tracks of a disk experience less wear and tear than the inner tracks due to lower rotational speed. This can lead to longer disk lifespan and reduced maintenance costs.
3. **Optimized Storage:** By focusing on the outer tracks, databases can optimize their storage capacity and organization, potentially reducing the need for additional storage resources.

S3. Assume that the access pattern to a relation's data is primarily sequential. How might the storage manager place the relation's data on a cylinder, head, sector storage device?

1. **Sequential Access:** Since the access pattern is primarily sequential, the storage manager would place the relation's data on a cylinder, head, sector storage device in a way that minimizes seek time.
2. **Cylinder:** The cylinder is the set of all sectors on a particular track. By placing related data on the same cylinder, the storage manager can reduce the number of head movements required to access the data.
3. **Head:** The head is the read/write head that reads and writes data to the disk. By placing related data on the same head, the storage manager can reduce the number of head movements required to access the data.
4. **Sector:** The sector is the smallest unit of storage on a disk. By placing related data on the same sector, the storage manager can reduce the number of seek operations required to access the data.

S4. List two advantages of solid-state storage/drives over magnetic disk drives/storage devices. What is a disadvantage of SSD relative to magnetic disk storage?

1. **Speed:** SSDs offer faster read and write speeds compared to magnetic disk drives. This is due to the absence of moving parts in SSDs, which eliminates seek times and reduces latency.
2. **Durability:** SSDs are more durable than magnetic disk drives. They are not susceptible to mechanical failures like head crashes, and are less prone to data corruption.
3. **Weight:** SSDs are lighter and more compact than magnetic disk drives, making them easier to transport and install.
4. **Noise:** SSDs produce less noise than magnetic disk drives, which is beneficial for applications that require a quiet environment.
5. **Power Consumption:** SSDs consume less power than magnetic disk drives, which can lead to longer battery life for laptops and other devices.
6. **Reliability:** SSDs are more reliable than magnetic disk drives. They are less prone to data corruption and have a longer lifespan.
7. **Cost:** SSDs are more expensive than magnetic disk drives, especially at higher capacities.

S5. What is an advantage of RAID-0 relative to RAID-5? What is an advantage of RAID-5 relative to RAID-0?

1. **RAID-0:** RAID-0 is a storage configuration that splits data across multiple disks in a way that allows for parallel access. This can lead to faster read and write speeds compared to RAID-5.
2. **RAID-5:** RAID-5 is a storage configuration that uses parity information to protect data from disk failures. This can lead to faster read and write speeds compared to RAID-0.

S6. Briefly explain logical block addressing and cylinder-head-sector block addressing.

1. **Logical Block Addressing:** Logical block addressing is a method of addressing data blocks on a storage device. It involves using a logical block number to identify the block of data on the device.
2. **Cylinder-Head-Sector Block Addressing:** Cylinder-head-sector block addressing is a method of addressing data blocks on a storage device. It involves using a cylinder number, head number, and sector number to identify the block of data on the device.

S7. Briefly explain the disk block access concepts of read-ahead and disk-arm scheduling.

1. **Read-Ahead:** Read-ahead is a technique used to improve the performance of disk access. It involves reading data from the disk into the buffer cache before it is needed, which can lead to faster access times.
2. **Disk-Arm Scheduling:** Disk-arm scheduling is a technique used to improve the performance of disk access. It involves scheduling disk operations in a way that minimizes seek time, which can lead to faster access times.

S8. Briefly explain the concepts of network addressable storage and storage area networks.

1. **Network Addressable Storage:** Network addressable storage is a type of storage that can be accessed over a network. It is typically used in storage area networks (SANs), which are networks of storage devices that are connected to servers and other devices over a network.
2. **Storage Area Network:** A storage area network (SAN) is a network of storage devices that are connected to servers and other devices over a network. SANs are typically used in data centers and other large-scale storage environments.

Storage Formats

F1. Assume the storage manager is using fixed size/length records. Explain two approaches to implementing addition/deletion of records.

1. **Free Space List:** The free space list is a list of free blocks of storage on the disk. When a record is added to the disk, the storage manager searches the free space list for a block of storage that is large enough to accommodate the record. The record is then added to the disk, and the free space list is updated to reflect the new free space.
2. **Bit Map:** A bit map is a bitmap that represents the free and used blocks of storage on the disk. When a record is added to the disk, the storage manager sets the corresponding bit in the bit map to indicate that the block is now used. When a record is deleted from the disk, the storage manager clears the corresponding bit in the bit map to indicate that the block is now free.

F2. VARCHAR is a common column type and is by definition not a fixed size field. Why might the storage manager choose to use fixed size records and allocate the maximal record size for records with VARCHAR fields?

1. **Fixed Size Records:** Fixed size records are easier to manage and can be more efficient in terms of storage and retrieval.
2. **Maximal Record Size:** Allocating the maximal record size ensures that there is enough space on the disk for the record, even if the VARCHAR field is not used to its full capacity. This can help prevent fragmentation and improve performance.
3. **Flexibility:** Fixed size records provide more flexibility in terms of record size. The storage manager can allocate the exact amount of space needed for the record, which can help prevent fragmentation and improve performance.
4. **Efficiency:** Fixed size records can be more efficient in terms of storage and retrieval. The storage manager can allocate the exact amount of space needed for the record, which can help prevent fragmentation and improve performance.
5. **Simplicity:** Fixed size records are simpler to implement and can be more efficient in terms of storage and retrieval.

F3. Consider the image below. Why does the storage manager consolidate free space by moving records when a deletion occurs?

1. **Consolidation:** Consolidating free space by moving records helps to prevent fragmentation. Fragmentation is the process of breaking up a large block of storage into smaller pieces, which can make it more difficult to manage and retrieve.
2. **Efficiency:** Consolidating free space by moving records can help to improve performance. By moving records to a single, contiguous block of storage, the storage manager can reduce the number of seeks required to access the data, which can lead to faster access times.
3. **Simplicity:** Consolidating free space by moving records is simpler to implement and can be more efficient in terms of storage and retrieval.

F4. Briefly explain head file organization, sequential file organization and multi-table clustering organization. Give a scenario or use case for choosing each of the approaches.

1. **Head File Organization:** Head file organization is a method of organizing data in a file by storing the records in a linked list of blocks. Each block contains a pointer to the next block in the list.

2. **Sequential File Organization:** Sequential file organization is a method of organizing data in a file by storing the records in a linear sequence. The records are stored in the order they were added to the file, and new records are added to the end of the file.
3. **Multi-Table Clustering Organization:** Multi-table clustering organization is a method of organizing data in a file by storing the records in a clustered index. The records are stored in the order they were added to the file, and new records are added to the end of the file.

F5. Consider a relation/file that records credit card transactions. Applications periodically retrieve records. 90% of the retrievals are for transactions that occurred in the prior six months. What technique would you recommend for storing the records on disks/files?

1. **Sequential File Organization:** Since 90% of the retrievals are for transactions that occurred in the prior six months, a sequential file organization would be a good choice. The records can be stored in a linear sequence, and new records can be added to the end of the file.
2. **Multi-Table Clustering Organization:** Since 90% of the retrievals are for transactions that occurred in the prior six months, a multi-table clustering organization would be a good choice. The records can be stored in a clustered index, and new records can be added to the end of the file.

Buffer Management

B1. Operating systems almost exclusively use the least recently used algorithm for the replacement policy. Why do databases sometimes use other algorithms? What is an example of a query for which most recently used might be a good replacement algorithm?

1. **Least Recently Used:** The least recently used algorithm is a good replacement policy because it is simple and efficient. It is also a good choice for most applications, as it balances the need for space with the need for performance.
2. **Most Recently Used:** The most recently used algorithm is a good replacement policy because it is simple and efficient. It is also a good choice for most applications, as it balances the need for space with the need for performance.
3. **Example:** A query for which most recently used might be a good replacement algorithm is a query that retrieves the most recently added records. In this case, the most recently used algorithm would be a good choice, as it would help to improve performance by keeping the most recently used records in the buffer.
4. **Other Algorithms:** There are other replacement policies that databases sometimes use, such as the first in first out (FIFO) algorithm and the random replacement algorithm. These algorithms can be more complex to implement, but they can also be more effective in certain situations.

B2. Briefly explain the Clock Algorithm for buffer replacement. What would motivate using the Clock Algorithm instead of least recently used?

1. **Clock Algorithm:** The Clock Algorithm is a replacement policy that is similar to the least recently used algorithm. It is also known as the circular linked list algorithm. The Clock Algorithm uses a circular linked list to keep track of the pages in the buffer. The algorithm starts at the head of the list and moves through the list, evicting the page at the head of the list when a page fault occurs.
2. **Motivation:** The Clock Algorithm is a good replacement policy because it is simple and efficient. It is also a good choice for most applications, as it balances the need for space with the need for performance.

B3. What is a pinned block? Does the buffer manager typically use pinned blocks when selecting a block for replacement? Why?

1. **Pinned Block:** A pinned block is a block in the buffer that cannot be evicted or replaced until it is unpinned. This is typically used for blocks that are currently in use by an application or process, ensuring that the data remains available for ongoing operations.
2. **Buffer Manager Behavior:** The buffer manager does not typically use pinned blocks when selecting a block for replacement. This is because pinned blocks are essential for maintaining data integrity and performance; evicting them could lead to data loss or increased latency for applications that rely on that data.

B4. Why would a buffer manager not necessarily write updated blocks to disk in the same order that the updates occurred in the buffer?

1. **Efficiency:** The buffer manager may choose to write blocks to disk based on factors such as the frequency of access or the size of the blocks, rather than the order of updates. This can optimize disk I/O operations and improve overall performance.
2. **Batching:** Writing multiple blocks at once can reduce the overhead associated with disk writes. The buffer manager may accumulate several updates and write them in a single operation, which is more efficient than writing each block immediately.
3. **Consistency:** The buffer manager may prioritize writing blocks that are critical for maintaining data consistency or integrity, regardless of the order in which they were updated. This ensures that important changes are preserved in the event of a failure.
4. **Asynchronous Writes:** Many buffer managers use asynchronous write operations, allowing updates to be queued and written to disk at a later time. This decouples the timing of updates in the buffer from the actual writes to disk, further complicating the order of operations.

Indexes

I1. How many clustered indexes can exist on a relation/table? Is it possible for a sparse index to be non-clustering/unclustered?

1. **Clustered Indexes:** A relation/table can have only one clustered index. This is because a clustered index determines the physical order of data in the table, and having multiple clustered indexes would create ambiguity in data storage.
2. **Sparse Index:** Yes, it is possible for a sparse index to be non-clustering/unclustered. A sparse index does not contain an entry for every search key value; instead, it only contains entries for some of the values. This can be implemented as a non-clustered index, where the index points to the actual data rows that may not be stored in a contiguous manner.

I2. Assume that a very common query on a table with information about people is `select * from people where last_name like "XYZ%"` where X, Y and Z are characters. For example, queries of the form `where last_name like "FER%"` or `last_name like "HAW%"` Can you use a hash index to optimize performance for this type of query?

1. **Hash Index Limitation:** A hash index is not suitable for optimizing queries that use the **LIKE** operator with a wildcard at the end (e.g., `last_name LIKE 'XYZ%'`). This is because hash indexes are designed for equality comparisons and do not support range queries or pattern matching.

2. **Alternative Indexing:** Instead, a B-tree index or a similar structure would be more appropriate for this type of query, as it can efficiently handle prefix searches and range queries, allowing for better performance when searching for last names that start with specific characters.

I3. If indexes are so effective at improving query performance, what is the disadvantage of creating very many indexes on a table?

1. **Increased Storage Requirements:** Each index consumes additional disk space. As the number of indexes increases, the storage requirements can become significant, especially for large tables.
2. **Slower Write Performance:** Every time a record is inserted, updated, or deleted, all relevant indexes must also be updated. This can lead to slower write performance, as the overhead of maintaining multiple indexes can be substantial.
3. **Complexity in Index Management:** Managing a large number of indexes can complicate database maintenance. It may become challenging to determine which indexes are beneficial and which are redundant or unused, leading to potential inefficiencies.
4. **Potential for Index Fragmentation:** With many indexes, there is a higher chance of fragmentation, which can degrade performance over time. Regular maintenance may be required to rebuild or reorganize indexes to maintain optimal performance.

I4. Data Structure courses teach binary search trees. It is common for the degree of a B+ tree to be more than 2. Why do B+ trees use degrees higher than 2? What determines the degree?

1. **Higher Degree for Efficiency:** B+ trees use degrees higher than 2 to increase the number of children per node, which reduces the height of the tree. This leads to fewer disk accesses when searching for data, as more keys can be stored in a single node.
2. **Balanced Structure:** A higher degree allows for a more balanced structure, which helps maintain efficient search, insert, and delete operations. This balance is crucial for performance, especially in databases where large amounts of data are handled.
3. **Determining the Degree:** The degree of a B+ tree is typically determined by the size of the nodes and the size of the keys. It is influenced by the block size of the storage medium, as the goal is to maximize the number of keys stored in each node while ensuring that the tree remains balanced.

Query Processing

Q1. Briefly explain this sentence, "Each relational algebra operation can be evaluated using one of several different algorithms." Give three examples for a select operation.

1. **Multiple Algorithms:** The statement means that for any given relational algebra operation, such as selection, there are various algorithms that can be employed to execute that operation. The choice of algorithm can affect performance based on factors like data size, indexing, and the specific database system in use.
2. **Examples for Select Operation:**
 - **Linear Search:** This algorithm scans each record in the table to find those that meet the selection criteria. It is simple but can be inefficient for large datasets.
 - **Indexed Search:** If an index exists on the column being queried, the database can use the index to quickly locate the relevant records, significantly improving performance compared to a linear search.

- **Bitmap Indexing:** This technique uses a bitmap for each distinct value in the column, allowing for efficient querying, especially in cases with low cardinality. It can quickly combine results from multiple conditions using bitwise operations.

Q2. Consider the following strange query – `select * from customers join employees on customers.last_name != employees.last_name`. Would an index nested loop join work for this type of join?

1. **Index Nested Loop Join Limitation:** An index nested loop join is typically effective for equality conditions (e.g., `customers.last_name = employees.last_name`). However, in this case, the join condition uses a non-equality operator (`!=`), which complicates the use of an index.
2. **Performance Considerations:** Since the join condition involves checking for non-matching last names, the index nested loop join would require scanning through all combinations of records from both tables, leading to potentially poor performance. This is because it cannot leverage the index efficiently to filter out matching records.
3. **Alternative Approaches:** A more suitable approach for this type of join might involve a full table scan or a different join strategy, such as a hash join, depending on the specific database implementation and the size of the tables involved.

Q3. What is the primary advantage of a merge join? If a merge join requires sorting the relations before performing the join, how is it possible better than a nested-loop join?

1. **Primary Advantage of Merge Join:** The primary advantage of a merge join is its efficiency when both input relations are already sorted on the join key. Merge joins can process the data in a single pass through each relation, making them very efficient for large datasets.
2. **Sorting Requirement:** While a merge join requires the relations to be sorted, this sorting can be done once and reused for multiple joins. If the data is frequently accessed in sorted order, the overhead of sorting can be amortized over multiple operations.
3. **Comparison to Nested-Loop Join:** In contrast, a nested-loop join examines every combination of records from both relations, which can be very inefficient, especially for large datasets. The merge join's linear scanning approach after sorting can significantly reduce the number of comparisons needed, leading to better performance in scenarios where the relations are large and sorted.

Q4. Consider the query `select first_name, last_name from person where residence_country='USA'`. What modification to the query might cause the query engine to create an index on `(first_name, last_name)`?

1. **Modification for Index Creation:** To encourage the query engine to create an index on `(first_name, last_name)`, you could modify the query to include a condition that filters based on `first_name` or `last_name`. For example:

```
SELECT first_name, last_name FROM person WHERE residence_country='USA'
AND first_name='John';
```

This modification provides a more selective condition that can benefit from an index on `(first_name, last_name)`.

2. **Composite Index Benefits:** Creating a composite index on `(first_name, last_name)` would allow the database to efficiently retrieve records that match both conditions, improving query performance, especially if the dataset is large.
3. **Database Optimization:** Additionally, if the database optimizer detects frequent queries that filter by both `first_name` and `last_name`, it may automatically suggest or create an index to optimize performance.

Q5. Briefly explain the concepts of pipelined evaluation and materialized evaluation.

1. **Pipelined Evaluation:** Pipelined evaluation is a technique where the output of one operation is directly fed as input to the next operation without waiting for the entire result set to be produced. This allows for continuous processing of data, reducing the need for intermediate storage and improving overall efficiency. It is particularly useful in query processing where operations can be performed in a streaming fashion, allowing for faster response times.
2. **Materialized Evaluation:** Materialized evaluation, on the other hand, involves executing a query and storing the complete result set in a temporary table or materialized view. This approach allows for faster access to the results since they are precomputed and stored, but it requires additional storage space and may involve overhead for maintaining the materialized data, especially if the underlying data changes frequently. Materialized views can be refreshed periodically to ensure they reflect the current state of the data.